

CS 411-W Lab 2: Individual

Brennen Gabriel

Team Gold, Old Dominion University

November 24, 2025

Table of Contents

1	<i>Introduction</i>	2
1.1	Purpose	2
1.2	Scope	2
1.3	Definitions, Acronyms, and Abbreviations	3
1.4	References	4
1.5	Overview	5
2	<i>Overall Description</i>	5
2.1	Product Perspective	5
2.2	Product Functions	5
2.3	User Characteristics	6
2.4	Constraints	6
2.5	Assumptions and Dependencies	6

No table of figures entries found.

1 Introduction

This document presents the Software Requirements Specification (SRS) for *Rogue-Like Algebra*, an educational game developed by Team Gold at Old Dominion University. The purpose of this document is to define the project's goals, system scope, terminology, and reference materials in accordance with the IEEE 830-1998 standard. It serves as a formal foundation for understanding the intended behavior, performance expectations, and constraints of the system. The contents that follow describe the overall purpose of the SRS, identify the target users and intended benefits of the software, and outline the structure of the remaining sections.

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the intended features, functions, and overall behavior of the *Rogue-Like Algebra* system. This document provides a detailed framework to ensure a mutual understanding between the development team and stakeholders regarding what the software must accomplish. It serves as a foundation for subsequent design, implementation, and testing phases, and acts as a baseline for verification and future enhancements.

1.2 Scope

Rogue-Like Algebra is a turn-based educational game designed to help students master algebraic concepts through interactive gameplay. The system integrates traditional rogue-like mechanics—such as procedurally generated dungeons and incremental difficulty—with mathematical problem-solving to make learning both engaging and personalized.

The project aims to improve students' confidence and persistence in mathematics by transforming abstract equations into game actions like attacking and defending. Its benefits include adaptive learning, immediate feedback, and a fun, replayable learning environment accessible through modern web browsers.

1.3 Definitions, Acronyms, and Abbreviations

1. **Adaptive Learning** – An educational method that adjusts content difficulty based on the learner's performance to personalize the learning experience.
2. **Algebraic Concepts** – Foundational mathematical principles involving variables, equations, expressions, and functions typically taught in middle and high school.
3. **Continuous Integration (CI)** – A development practice where developers frequently merge code changes into a shared repository, triggering automated builds and tests to detect issues early.
4. **Continuous Deployment (CD)** – A software release process that automatically delivers validated code changes to production, minimizing manual intervention and ensuring faster updates.
5. **GScript** – A Python-like scripting language used in the Godot game engine, designed for creating game logic.
6. **Git** – A distributed version control system that tracks changes in source code during software development, enabling collaboration and history tracking.
7. **GitHub** – A cloud-based platform for hosting Git repositories. It offers tools for collaboration, issue tracking, code review, and CI/CD integration.
8. **Godot** – A free and open-source game engine used for developing 2D and 3D games, known for its flexibility and ease of use.
9. **IDE (Integrated Development Environment)** – A software suite that provides tools like a code editor, debugger, and compiler to streamline software development (e.g., VS Code).
10. **Procedural Generation** – A method of creating data algorithmically rather than manually, often used for generating levels, puzzles, or content dynamically in games.
11. **Rogue-like** – A subgenre of games typically featuring procedural generation, turn-based movement, and permanent death, emphasizing strategy and replayability.
12. **Rust** – A systems programming language known for performance, safety, and concurrency.
13. **Turn-Based Combat** – A gameplay mechanic where players and opponents take alternating turns to perform actions such as attacking or defending.

14. **VS Code (Visual Studio Code)** – A popular, lightweight code editor developed by Microsoft, used for writing and debugging code.

1.4 References

1. “About Us - DragonBox / WeWantToKnow.” Dragonbox.com, dragonbox.com/about.
2. Prodigy. “Prodigy Math | Prodigy Education.” Www.prodigygame.com, www.prodigygame.com/main-en/prodigy-math/.
3. “IXL - Membership.” Www.ixl.com, www.ixl.com/membership.
4. Steam. “Steam Database.” SteamDB, steamdb.info/.
5. “Top Roguelike Games by Sales and Revenue on Steam as of 2022 – Steam Marketing Tool.” Games-Stats.com, games-stats.com/steam/?tag=roguelike.
6. Decker-Woodrow, Lauren E, et al. The Impacts of Three Educational Technologies on Algebraic Understanding in the Context of COVID-19. Vol. 9, 1 Jan. 2023, p. 233285842311659-233285842311659, <https://doi.org/10.1177/23328584231165919>.
7. Aine, Sandip, and Maxim Likhachev. “Anytime Truncated D* : Anytime Replanning with Truncation.” Proceedings of the International Symposium on Combinatorial Search, vol. 4, no. 1, 20 Aug. 2021, pp. 2–10, <https://doi.org/10.1609/socs.v4i1.18295>
8. Guy, Stephen, and Ioannis Karamouzas. Guide to Anticipatory Collision Avoidance.
9. “NavigationMesh.” Godot Engine Documentation, 2025, docs.godotengine.org/en/stable/classes/class_navigationmesh.html
10. “Creating Your First 2D Game with A* Algorithm.” HackerEarth Blog, 6 Oct. 2016, hackerearth.com/blog/developers/creating-first-2d-game-algorithm/
11. Team Gold. (2025, October 10). *Lab 1 – Rogue-Like Algebra Product Description*. Old Dominion University. Retrieved October 29, 2025 from <https://gold-team-odu.github.io>

1.5 Overview

Section 2 provides an overall description of the product, covering the system's perspective, major functions, and target users. Section 2.1 describes the product's perspective and how Rogue-Like Algebra fits into the overall system environment. Section 2.2 summarizes the major functions provided by the system, and Section 2.3 identifies the intended users and their characteristics. Section 2.4 outlines the general constraints that may affect design decisions, and Section 2.5 explains the assumptions and dependencies relevant to this project. The remainder of this document specifies the functional requirements and system attributes for the prototype version of Rogue-Like Algebra in accordance with the IEEE 830-1998 standard.

2 Overall Description

This section provides a general overview of the *Rogue-Like Algebra* system and the factors that influence its requirements. It outlines how the product fits within its environment, summarizes its main features and intended users, and identifies any assumptions or constraints that shape development. Rather than describing individual functional requirements, this section focuses on the system's context and purpose to help readers understand the reasoning behind the detailed specifications presented later in this document.

2.1 Product Perspective

Rogue-Like Algebra is an independent, web-based educational game that integrates algebraic problem-solving with turn-based rogue-like gameplay. The system relies on the Godot Engine for rendering and scene management, with Rust handling core logic and GDScript used for scripting game-specific interactions.

The product is designed to run in modern browsers on Windows, Linux, and macOS systems, requiring no external dependencies beyond the web platform. Core components include the procedural generation system, combat and problem-solving modules, adaptive difficulty engine, and user interface elements such as menus, HUD, and feedback displays.

2.2 Product Functions

Major functional capabilities of *Rogue-Like Algebra* include:

- **Dynamic Problem Generation:** Randomized algebra questions tied to game actions such as attacking, defending, or solving puzzles.
- **Adaptive Difficulty:** Adjusts problem complexity based on player accuracy and progression.

- Turn-Based Combat System: Uses mathematical accuracy to determine the success of player moves.
- Procedural Dungeon Generation: Creates unique map layouts and challenges for each play session.
- Feedback and Rewards System: Reinforces learning through experience points, upgrades, and in-game achievements.

These core functions align learning objectives with gameplay incentives, ensuring engagement while reinforcing algebraic principles.

2.3 User Characteristics

The primary users are middle- and high-school students (ages 12–18) who are learning or reinforcing algebraic concepts. They may possess beginner-to-intermediate math proficiency but are familiar with basic computer and gaming interfaces.

Secondary users include teachers, parents, and educational facilitators who monitor student progress or use the game as supplemental instruction. The interface emphasizes clarity and ease of navigation, minimizing text complexity and supporting visual learning through interactive gameplay.

2.4 Constraints

This section is N/A per Lab 2 guidelines.

2.5 Assumptions and Dependencies

This project assumes that users have access to a modern web browser and a reliable internet connection. Development depends on continued compatibility between the Godot Engine and its Rust bindings.

